

A Canary in the Crypto Mine: Defeating Stack Protection in a GPU Secure Coprocessor

Jon Pry
jonpry@gmail.com

June 2026

Abstract

A widely sold datacenter-class accelerator (the NVIDIA “CMP 170HX”) is the *same die* as a flagship A100 but is fuse-crippled on three commercial axes: SM math rate (throttled to 1/32), memory capacity (10 GB instead of 80 GB), and PCIe link (Gen1 instead of Gen4). We show that all three caps are software-defeatable from a rooted-but-otherwise-unprivileged host, with no signing key, no debug fuse, and no physical access. The root enabler is arbitrary code execution in the GPU’s Heavy Secure (HS) Falcon mode. The HS break does not depend on the value of the stack canary: the stack-protector’s *reference word* lives in writable memory reachable by the same data overflow it is meant to detect, so secrecy and entropy are irrelevant. Because the microcode is deterministic, we emulate it offline and author an overflow payload that is a *valid continuation image*, so the victim function passes its own canary check and returns into our chain. The economic consequence is that differentiation the business model treats as hardware-permanent is, for the three axes we defeated, only firmware-gated. Three further axes (PCIe Gen3, on-die ECC, runtime HBM retiming) resisted our attempts; we report this without claiming they are truly unbreakable, since we do not fully understand the relevant fuses or initialization. The HS code-execution bug is a **regression** present in the newer open-kernel-modules-era booter and absent in an older one, so version currency is not a security gradient, and an open host driver did not imply an audited signed booter. Measured gains: roughly 31–62x compute, 8x capacity, 2x link. We distill four transferable findings.

1. Introduction

Modern silicon vendors sell one die as many SKUs. The price delta between a flagship accelerator and a cut-down “mining” or “inference” part of the same die is, increasingly, not enforced by physically different silicon but by *fuses* and by *firmware that reads those fuses* and decides which features to enable. This raises a security question that is usually left implicit: what is the real boundary protecting that price discrimination, and is it as permanent as the business model assumes?

The implicit assumption stack has three layers. (a) Fuses are permanent and authoritative. (b) The firmware that enforces them runs in a hardware-isolated secure mode, on these GPUs, Heavy Secure (HS) microcontroller mode behind write-protected memory regions. (c) That secure mode is itself protected by signature verification *and* by ordinary memory-safety mitigations: priv-level masks on registers, and stack canaries inside the secure code. We show that layer (c) is the weak link, and that breaking it dissolves (a) and (b) for every axis not *additionally* backed by a physical gate.

Getting into HS required breaking four assumptions that defenders routinely rely on. They form the spine of the paper, and each one generalizes beyond this particular card:

Assumption defenders rely on	Why it fails here	Sec.
Signature gating keeps untrusted code out of HS	the bug is inside the signed code, reached after the check has already passed	§5.4
Canary strength = secrecy / entropy	wrong axis; the <i>reference word</i> is mutable, so its value is irrelevant	§5.1–5.3
You can corrupt memory but not reconstruct a coherent continuation	deterministic microcode is fully emulatable, so the payload <i>is</i> a valid state image	§5.5
DMA’d control data is part of the analyzable signed image	post-DMA reachability is opaque to static analysis, and it hides bugs from the vendor too	§3.2

Contributions.

1. **A stack-canary defense in HS firmware is defeated despite being implemented to best practice.** The master stack guard is *reseeded every boot from the hardware secure RNG*, with maximal entropy, fresh per boot, never a constant. It still provides zero protection, because the toolchain places the guard in writable memory reachable by the very data-section overflow it is meant to detect, and the function epilogue *re-reads* that global. A single uniform-fill overrun sets the guard, the saved on-stack copy, and the return address all to one adversary value V , so the check is $V == V$ (passes) and the return diverts to V . Randomness, secrecy, and per-boot freshness are all irrelevant: the mitigation reduces to “is the reference word immutable?”, and the default toolchain placement answers “no.”
2. **Target determinism is an exploitation oracle that defeats runtime unobservability.** In HS mode the secure Falcon’s instruction and data memory are locked from any external read, so the adversary is blind to the live stack and has only a single mailbox register as feedback. Because the microcode has no concurrency, interrupts, OS, or allocator nondeterminism, we instead emulate it cycle-faithfully, reconstruct the exact memory state at any program point offline, and craft an overflow payload that is a *valid continuation image*. Minimal-firmware simplicity is here the adversary’s chief asset.
3. **DMA-opaque control flow is a structural blind spot** for static analysis, for the vendor auditing its own signed image as much as for us as reverse engineers.
4. **End-to-end de-differentiation**, with a taxonomy separating what we *demonstrably* defeated from what *resisted the attempts we made*. We defeated the SM math rate, the memory capacity, and the PCIe link up to Gen2. We did **not** find a software path to PCIe Gen3, to on-die ECC, or to runtime HBM mode-register programming, but we do not claim these are impossible: in each case we do not fully understand the relevant fuse semantics or initialization sequence, so “we found no path” is a statement about our effort and understanding, not a proof that none exists.

Scope and ethics preview. This is single-card lab work, conducted under responsible disclosure, with no persistent silicon change and no extracted signing key. We disclosed the core defect to the vendor’s product-security team. Section 10 details the framing.

2. Background

2.1 GPU product differentiation via fuses

These accelerators carry one-time-programmable (OPT) fuses that are *sensed at boot* and exposed to firmware as read-only capability words. For each commercial axis there is a corresponding *feature-override shadow register*: a writable register that, when its priv-level mask permits, overrides the fused cap. SM math rate, framebuffer geometry and strap selection, and a PCIe boot-speed (Gen3-disable) bit are all expressed this way.

The distinction that matters here, and that the rest of the paper depends on, is between a *fuse read by firmware* and a *fuse that physically gates a datapath*. The former merely tells the firmware what to advertise

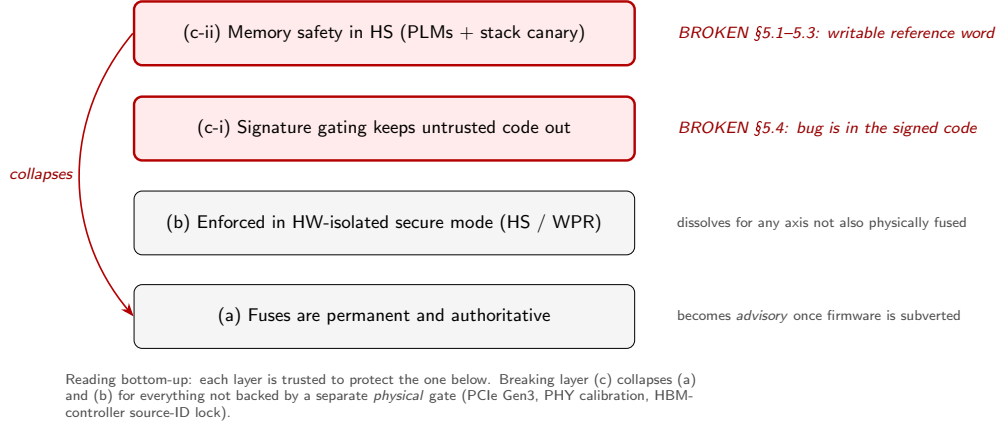


Figure 1: The trust / assumption stack protecting product differentiation. Each layer delegates to the one below; the highlighted links are the assumptions this paper breaks (Section 5). Collapsing the secure-mode foothold dissolves the layers above it for every axis not also backed by a physical gate.

and enable; if the firmware can be subverted, the cap is advisory. The latter actually disconnects circuitry (an unprovisioned PHY calibration profile, a disabled ECC datapath) and remains permanent regardless of firmware. The card’s differentiation mixes both tiers, and the security boundary is only as strong as the weakest “firmware reads a fuse and decides” check.

2.2 NVIDIA secure-firmware architecture

The GPU contains several microcontrollers (Falcon and NV-RISCV cores) that run signed firmware [12, 16]: the GSP (the offloaded resource manager), SEC2 (the security engine), FBFLCN (the framebuffer/HBM controller), and PMU (power). Three mechanisms form the defender’s defense-in-depth.

Priv Level Masks (PLM). Each security-critical register carries a per-register access-control mask that gates reads and writes by the requesting agent’s privilege level and source identity. This is the layer that actually stands between a host process and the fuse-override shadow registers.

Heavy Secure (HS) mode and the boot ROM signature check. A Falcon can enter HS mode only after the immutable boot ROM verifies an RSA-3K signature over the code image. HS code runs at the highest privilege level (LEVEL2) and can rewrite PLMs and program the protected regions.

ACR / WPR and the FBHUB ACL. Write-Protected Regions in the framebuffer are carved by the Access-Control Region bootstrap; a source-ID / priv-level access-control list in the framebuffer hub enforces who may touch them. GSP-RM, the resource manager running on the GSP, exposes a large RPC surface (RmCtrl) to the host driver, layered on top of all of the above.

2.3 Threat model

This threat model is inverted relative to the usual one. The host is *not* an asset to be protected: it is fully owned by the adversary, who has root, controls the kernel, and can map and drive the device at will. The defender is the *GPU itself*: its firmware, acting for the vendor, tries to keep behavior the vendor has fused off out of reach of the machine’s own owner. The secured asset is the card’s fused configuration; the adversary is the legitimate owner of the card. We use “secure,” “privileged,” and “trusted” throughout from the GPU’s point of view: the host is the lowest-privilege, untrusted agent, and the secure domain is the on-chip state the firmware defends against it.

Concretely, the adversary is a privileged host process that maps the device’s PCI BAR0 aperture (for example through the kernel’s `resource0` sysfs node) and can read and write the chip’s registers. We assume no physical access, no JTAG, no debug fuse, no vendor signing key, and no silicon edit; the firmware is stock

and signed. The goal is to cross the host $\rightarrow$ GPU-secure-domain boundary; that is, to get the GPU to do what its owner asks despite the firmware’s attempt to refuse.

3. Methodology: Navigating a Signed Blob

3.1 Binary RE pipeline

We reverse-engineered the firmware images with a headless disassembler driven by scripted annotation, cross-verifying findings across a range of driver versions, examined to varying depth, from the 450 branch through 580 (see Section 7.2 for the corpus), to separate stable structure from per-version noise. Live ground truth came from poking PRI registers through the host BAR0 mapping and from a driverless Falcon non-secure-execution primitive that loads and starts small payloads on the engines without the vendor driver.¹

3.2 DMA defeats static reachability (Thesis 3)

The recurring structural finding is that the security-relevant decision is made on data the engine *DMA*s into its on-chip data memory (DMEM) and validates in place; the host-side system- memory copy is inert and consumed one-shot. This has three consequences.

For the *auditor*, static cross-reference sweeps terminate at the DMA boundary. One cannot prove that a handler is reachable, or what it consumes, without modeling the runtime transport. Our own worked example was a control RPC whose producer hunt dead-ended on a symbol-stripped object: the data path existed only after a DMA we could not statically follow.

The same property that hides reachability from us also hides bugs from the vendor. Signed-image static review systematically under-counts the attack surface that materializes only after a DMA. An image can pass a thorough static review and still ship a memory-safety defect, because the dangerous write is performed by a device engine rather than by a CPU instruction the analyzer models.

The regression we describe in §7.2 is itself evidence for this. We do not have to argue that static tooling misses DMA writes; the vendor demonstrated it. An unbounded DMA-copy helper invoked with an adversary-controlled signature-length field, overrunning a fixed buffer into the canary and saved return address, is exactly what a DMA-aware analysis would flag. That it shipped in newer code is a natural experiment showing that the vendor’s analysis pipeline does not model the DMA engine as a write primitive. The thesis is supported by the vendor’s own miss, then, and not only by our exploitation of it.

3.3 Live oracles

Where a static fact is unreachable behind a DMA, “the register either sticks or it doesn’t” is ground truth. A write that takes (reads back changed) versus one that is dropped, or a benchmark that jumps versus one that does not, substitutes for an unprovable static reachability argument.

3.4 The emulator as a state oracle (Thesis 2)

We built a cycle-faithful Falcon emulator whose purpose is not speed but *ground-truth memory state at an arbitrary program counter* for an opaque signed image. The reason it is *necessary*, not merely convenient, is the second blind. In HS mode the secure Falcon’s data and instruction memory are locked from any external read: the host cannot inspect the live stack, the guard word, or any local at the moment of corruption. The only runtime feedback channel is a single mailbox register, which carries almost no information (one value means the canary check failed, another means our spin loop is running). There is no live introspection with which to discover the values a continuation payload must carry.

¹We ran the lab harness under QEMU/VFIO passthrough purely for stability and host isolation, since a wedged GPU resets the guest, not the workstation. Virtualization is not part of the attack surface: register access comes from owning the PCIe function, so the attack runs identically on bare metal.

The emulator substitutes for that blindness. It reconstructs the exact data-memory and stack image offline; the mailbox register on silicon then *confirms* the prediction. Paired with §3.2, the two blinds are complementary: DMA hides reachability from static analysis, and HS locks memory from dynamic observation. The emulator is the only tool that recovers the full state under both. More generally, deterministic firmware is exhaustively emulatable, and that emulatability is what makes precise overflow-continuation authoring tractable when the live target cannot be observed.

4. The Register-Access Model and the Privilege Boundary

The GPU exposes its control registers as a memory-mapped PCI base-address region (BAR0). A userspace process that owns the device maps this region (for instance by `mmap`-ing the kernel's `/sys/bus/pci/devices/<bdf>/resource0` file, or through a VFIO device node) and obtains a pointer through which the chip's PRI registers are read and written as ordinary 32-bit loads and stores. No firmware RPC or driver call is involved; register access follows directly from mapping the BAR. The question that matters is therefore not whether the host can reach the registers (it always can) but which registers the *GPU* will let it change.

The firmware enforces that distinction through the priv-level mask (PLM) layer. Each register the GPU protects carries a PLM that admits writes only from agents the firmware considers sufficiently privileged, keyed on privilege level and source identity. The host is treated as the lowest privilege level (PL0). The registers that select product-differentiation behavior (the SM-speed override, the fuse-control register, the framebuffer-configuration registers, and the XVE/PCIe registers) are all PLM-protected against PL0 writes. The firmware usually still permits the host to *read* them; it is the writes that would change fused behavior that it refuses. The host can thus observe these registers but cannot, from PL0, make the GPU honor a write to the ones that matter.

The boundary the rest of the paper must cross is therefore narrow and specific: from PL0 (full BAR0 read/write, but PLM-denied on the differentiation gates) to the secure-domain privilege (Heavy Secure, LEVEL2) that is permitted to reprogram PLMs and carve the write-protected regions. Section 5 obtains that privilege; Section 6 shows what it makes reachable.

5. Collapsing Heavy Secure Mode

This section establishes Theses 1 and 2: a best-practice stack canary that provides zero protection because its reference word is writable, and a deterministic target whose emulatability lets us author a valid post-overflow continuation against an otherwise unobservable secure core.

5.1 What stack-smashing protection actually guarantees

The compiler's stack-smashing protection (SSP) [6] emits, in each protected function's prologue, `canary = *guard_global`, and in the epilogue *re-reads* the same global and compares it to the saved on-stack copy, branching to an abort routine on mismatch. SSP is sound only if the reference word stays unreachable and immutable for the duration of the call, a precondition that is rarely stated explicitly. Most work on canaries concentrates on the entropy of the value (whether it is random, secret, or brute-forceable) and pays little attention to whether the reference word can be written, which is the property that matters here.

5.2 The canary is strong; the reference word is writable

This is not a weak-canary story. The guard global is reseeded during HS init from the hardware secure RNG, giving a per-boot, high-entropy value, which is exactly what SSP is supposed to do. (In our emulator the seed surfaces as a fixed constant only because the crypto coprocessor is stubbed; on silicon it is a true hardware-random word. The mechanism of reseed-then-overflow is identical regardless of the value, and we never need to learn it.)

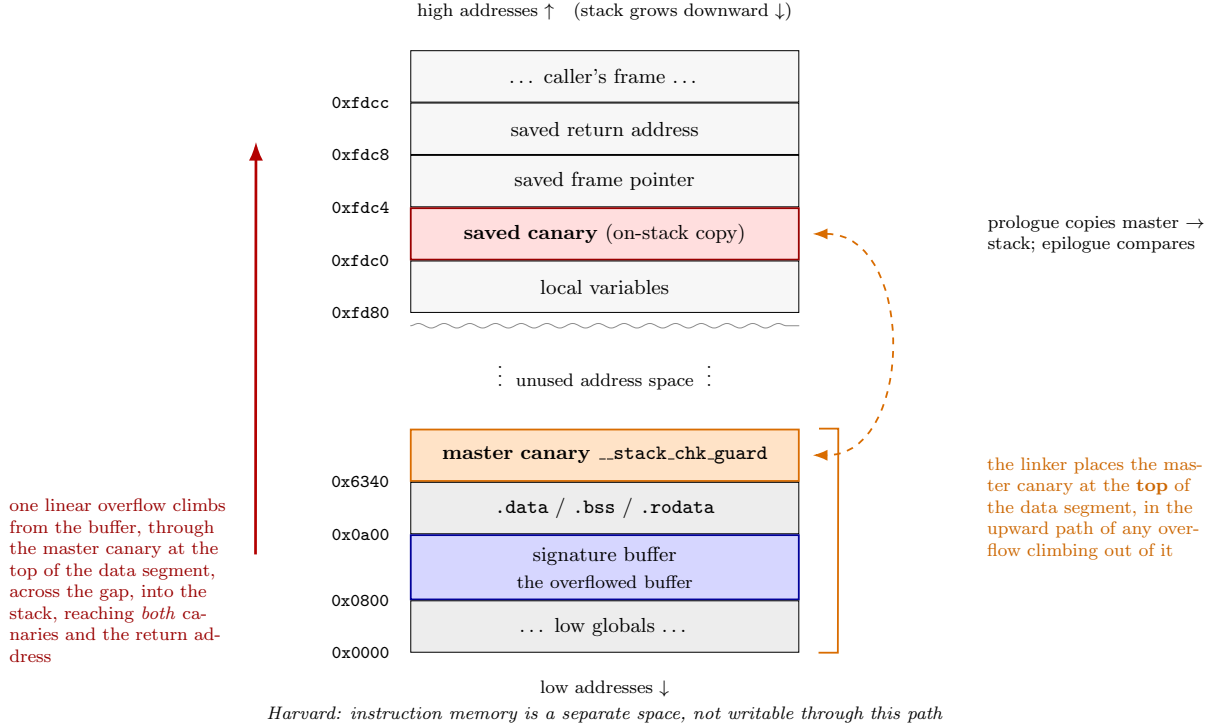


Figure 2: The two copies of the canary in *data* memory (illustrative addresses; this is a Harvard machine, so instruction memory is a separate space and is not writable through this path). The *master* guard is placed by the linker at the *top of the data segment*; the *saved* copy lives in the stack frame between the locals and the saved return address. The prologue copies master to stack and the epilogue re-reads the master and compares. The placement is the flaw: because the master sits at the top of the data segment, a single linear overflow out of a data-segment buffer climbs through it, crosses the gap into the stack, and reaches the saved copy and return address, so one overflow hits *both* canaries. The adversary normalizes them to one value without ever learning it; integrity of the reference word, not secrecy, is what fails.

What fails is where the guard is stored, not what it contains. The toolchain emits it at the tail of the read-only-data section, and in this flat, MPU-mapped Falcon image that region lies inside the writable data span: there is no read-only-after-init relocation hardening, no guard page, and no MPU read-only mapping. The guard therefore sits among ordinary globals that are themselves buffer-overflow targets. The word whose integrity the whole mitigation depends on is placed exactly where a linear data overflow lands, so the adversary never has to learn the random value; they overwrite it.

5.3 The defeat, value-independent

The defeat takes two steps. First, a data-section overflow writes `guard_global ← V` for an adversary-chosen `V`, so the canary is now known by construction, regardless of any randomization. Second, the same linear overflow writes the saved canary slot to `V` and the control state beyond it. The epilogue re-reads the poisoned global, compares `V == V`, passes, and returns into our chain.

Notice that this needs no leak, no brute force, and no prediction. The classic canary attacks, such as leaking the value and replaying it or brute-forcing it through a forking server, all aim at recovering the value, and none of them applies here. Whether the canary is public, constant, or freshly random makes no difference; the placement defeats it either way.

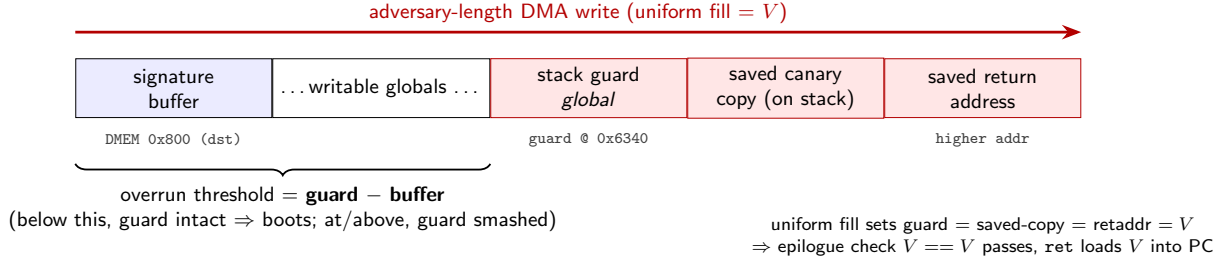


Figure 3: Data-memory layout at the overrun. A single unbounded copy with adversary-controlled length runs from the fixed signature buffer through the stack-guard global, its saved on-stack copies, and the saved return address. The overrun threshold is the *relative* distance guard – buffer; a uniform fill sets guard, copy, and return address to one value, so the epilogue check passes and the return diverts to it.

5.4 Signature is not the only gate

The boot ROM RSA-3K check guards *entry* to HS, and we never defeat it: we do not forge a signature. We load the vendor’s own signed booter, which passes the check and begins executing in HS. The flaw we exploit is a memory-safety bug inside that signed code, reached only after the signature has already been verified, so the signature check never gets the chance to stop us. A driverless userland loader (host BAR0, no vendor driver) supplies the signed booter together with the data the bug consumes; the signed code runs and we corrupt its state while it executes.

5.5 Authoring a valid-continuation overflow (Thesis 2)

The concrete bug is a single linear primitive. The LS signature-verification routine issues a DMA copy into a fixed low-memory *signature buffer* with a length taken from an adversary-controlled WPR-metadata field (the signature-length field), and the destination buffer is **unbounded**. The DMA copies the adversary’s signature blob verbatim from system memory, so the bytes that land on the canary, on the saved canary copies, and on every saved return address are all *adversary-chosen*.

A naive overflow fills these with garbage and trips the abort routine, a denial of service. The emulator (§3.4) gives us the exact set of words the overrun spans: the guard global, the signature-verifier’s own saved canary copy, and the saved return PC, in one contiguous write. We then make the payload a **valid continuation image**: a *uniform fill* with one value V simultaneously sets guard = saved-copy = return-address = V . The epilogue check becomes $V == V$ (passes), and the first return loads V into the program counter. Choosing V to be the address of an adversary-controlled instruction-memory gadget converts the denial of service into arbitrary HS program-counter control, with no crash, no watchdog trip, no leak, and no brute force.

The emulator is what lets us *pick a working V* . At runtime the HS Falcon’s data memory is unreadable from the host, so we cannot observe the live stack to learn which words the overrun must preserve and which it may divert. The emulator supplies that image offline; the silicon mailbox register is the only confirmation channel back. The uniform-fill V is the special case that needs *no* per-word reconstruction, because every clobbered word collapses to the same value. A *productive* return-oriented continuation [8], one that must leave selected locals intact, depends entirely on the emulator-derived state, because the live values can never be read.

Real emulator trace. The following is from our Falcon emulator running the GA100 booter image. Concrete addresses are retained here only because the mechanism is the relative relationship between them; the prose above uses roles.

```
# (A) naive / non-uniform signature, length 0xf800 -> DoS
REACHED SIG DMA: buffer=0x800 size=0xf800 overrun-end=0x10000 guard@0x6340
SIGSZ=0xf800 pc=0x7def spin=0x7def CANARY=True MB0=0x47 # stack-check-fail abort

# (B) uniform fill V=0x4a7 (address of an IMEM self-loop gadget) -> HS PC control
```

```

staged signature = 0x4a7 repeated; DMA smashes guard/canaries/return-addrs to 0x4a7
SIGSZ=0xf800 pc=0x4a7 spin=0x4a7 CANARY=False MB0=0x31 # canary PASSED, PC hijacked to V

# Guard word watch during run (B):
pc=0x04cc guard: 0x00000000 -> 0xc0deca7e # secure-RNG seed installed at HS init (stub value)
pc=0x05ba guard: 0xc0deca7e -> 0x000004a7 # sig-DMA overrun overwrites guard with adversary V

# Threshold dichotomy (matches the hardware length sweep):
size=0x5b00 overrun-end=0x6300 (< guard 0x6340) guard intact CANARY=False MB0=0x96 (boots on)
size=0x5c00 overrun-end=0x6400 (>= guard 0x6340) guard smashed CANARY=True MB0=0x47 (fail)

```

Listing: emulator trace, runs A and B and the threshold bracket.

The trace confirms each part of the argument. The guard is runtime-seeded from the hardware secure RNG (it goes from zero to the seed at one program point, a per-boot fresh value), yet its value does not matter, because the overrun rewrites it a few instructions later. The threshold is mechanical: the first length that reaches the guard is guard minus buffer, and the emulator’s boundary (0x5b00 boots, 0x5c00 fails) brackets it exactly. The uniform fill is the emulator-derived valid continuation that flips the mailbox from the abort code to our spin code with the program counter equal to V.

Silicon cross-check (recorded in prior work; not re-run in this drafting pass, per our no-hardware constraint). The same primitive ported to hardware yields the HS-mode status bit set, the spin mailbox code, and the CPU spinning at V; a different V halts instead. Spin-versus-halt is selected purely by the adversary’s V, giving arbitrary HS program-counter control on real silicon, confirming the emulator model.

The same technique should apply to other deterministic embedded or secure-coprocessor targets. Three ingredients combine to give reliable secure-mode (LEVEL2) code execution: a canary whose reference word can be overwritten, a uniform overflow value that passes the check regardless of what the canary holds, and an emulator that reconstructs the surrounding state so the corrupted function still runs to its return. That LEVEL2 privilege is what then lets us reprogram the PLMs and carve the protected regions.

The live exhibit above is the SEC2 booter image. The resource manager’s own resident blob carries a second instance of the same writable-reference-word class, a guard global that is itself a public hardcoded constant in that image. We use the booter primitive as the emulated and verified exhibit, and cite the resource-manager guard only as a second occurrence of the class in a different image.

6. De-Differentiation: From HS to SKU Unlock

Once we have HS code execution, the secure foothold does one thing for us: it opens the PLMs that gate the fuse-override shadow registers. After that, ordinary PL0 host BAR0 writes (§4) drive the overrides, and no further exploit is needed. A single bring-up recipe performs the steps; below we describe each axis by the control register it exposes and omit the full register map.

6.1 PLM unlocking as the pivot

The HS chain opens the priv-level masks on the SM-rate, framebuffer, and XVE/PCIe fuse-override registers. The operationally important result is that the unlock persists across a function-level reset (FLR). The override values we write through the opened PLMs are held in an always-on island and survive the reset, so the card stays in its unlocked configuration across an FLR and a clean stock driver load, with no need to re-run the exploit, even though the PLMs themselves may re-lock. A transient exploit thus leaves the card in a durable unlocked state that needs no exploit to maintain, convenient for the adversary and awkward for the defender (we return to this in §8.5).

6.2 SM throughput

The SM-rate lever is the *feature-override shadow register* for per-pipe math rate, which encodes rate codes from 1/32 up to full speed. With the fuse PLMs open, driving it to full speed restores real compute. Measured with a standard GEMM stress benchmark (gpu-burn [14]): FP32 ~31×, FP64 ~59–62×, and TF32

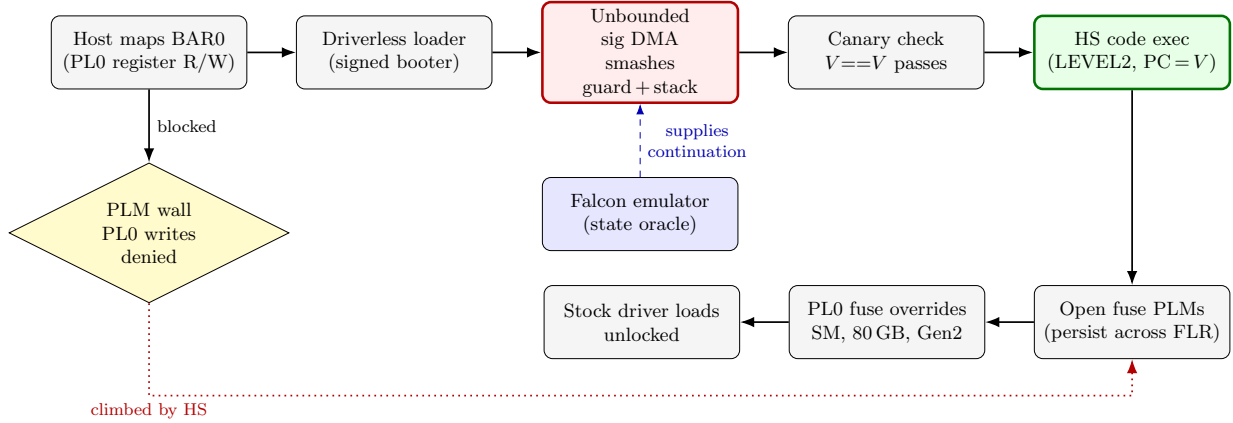


Figure 4: End-to-end attack flow. A privileged host can already read/write registers, but the fuse-override writes are denied at its privilege level (the PLM wall). A driverless loader runs the signed booter; the emulator supplies a valid continuation image for the unbounded-DMA overflow, which passes the canary check and yields secure-mode code execution. That foothold opens the fuse-override privilege gates, after which ordinary host register writes apply the overrides. The override values persist across reset, so the stock driver then loads against an already-unlocked card.

tensor $\sim 15\times$ (reaching ~ 94 TFLOPS). It works both written live (no reload) and set before boot, and it survives a driver reload. Peak temperatures under full-speed load were safe ($\leq 67^\circ\text{C}$; the full-capability part throttles only above $\sim 85^\circ\text{C}$). Since the fuse only lowers the rate and the override raises it back, the cap is a software-readable shadow rather than a physical limit.

6.3 Memory capacity and the stability problem

Exposing the full memory is, once the FB PLMs are open, a shadow-register write that changes the framebuffer geometry from the native 10 GB to 80 GB. The capacity is physically present: we wrote a distinct hash pattern across the full 80 GB and read it back intact, which confirms that all of the memory is real and addressable rather than aliased back onto a smaller region.

Stability at full capacity is a separate question. Under a full-speed memory stress test the 80 GB configuration produced errors (2,796 in one gpu-burn run) while the same card ran cleanly at 40 GB. A part that physically holds 80 GB but only passes stress testing at half capacity is consistent with silicon that failed quality control at full capacity and was binned down to the crippled SKU.

The instability is addressable from software. Refreshing the DRAM more often (lowering the refresh-interval field of one FB-config register) made the 80 GB configuration stable, with 0 errors in the same stress test, at a cost of about 32% in throughput (64.6 versus 94.6 TFLOPS), because the extra refresh traffic leaves less bandwidth for memory accesses. We did not sweep the other timing parameters; relaxing some of them to add margin might restore 80 GB stability at a smaller cost than the refresh change alone. We do not, however, claim a full-performance fix is available. If the part is genuinely a QC reject, the binding limit is the memory itself, and no choice of timings will give it the headroom a part that passed would have had. Tuning trades performance for stability margin; it does not turn a binned die into a good one.

We did not manage to program the HBM mode registers at runtime, which would be the clean way to retime the die and the apparent route to enabling on-die ECC. Every agent we could drive (the host, SEC2 in non-secure mode, SEC2 in HS, the PMU, and the GSP in non-secure mode) failed to issue a mode-register-set command that the die consumed. The writes appeared to be dropped on a source-identity basis, and the only agent that visibly drives the die is the FB Falcon running its own code at boot. This is strong evidence that the obvious runtime paths are gated, but it is not a proof that no path exists: we do not fully understand the controller’s source-ID checks or the boot initialization, and a vantage we did not try, or an initialization

Table 1: What the firmware break defeated, what a board rework defeated, and what resisted our attempts. The top three rows are demonstrated firmware defeats. Lane width is a confirmed *physical* gate that lies outside the firmware threat model: the upper lanes’ AC-coupling capacitors are simply depopulated, and we restored a full $\times 16$ link by soldering them on. The bottom two rows are axes we did *not* crack; their enforcement column is our *working hypothesis* for why each resisted, not established mechanism, so they remain open questions rather than proven walls.

SKU axis	Defeated?	Enforcement (demonstrated / apparent)
SM math rate (1/32 $\rightarrow$ full)	✓	fuse shadow read by firmware
Memory capacity (10 $\rightarrow$ 80 GB)	✓	FB geometry shadow
PCIe Gen1 $\rightarrow$ Gen2	✓	XVE fuse-override of link cap
Lane width ($\times 4 \rightarrow \times 16$)	HW rework	confirmed physical: upper-lane AC-coupling caps depopulated; restored by soldering
PCIe Gen3	not yet	no override field <i>found</i> ; PHY calibration <i>appears</i> fuse-gated
On-die ECC / runtime HBM MRS	not yet	HBM controller <i>appears</i> source-ID-locked

detail we misread, could change the conclusion. We report it as a limit of our effort.

6.4 PCIe link speed

The XVE *fuse-override* register moves the read-only advertised link capability (Gen1 $\rightarrow$ Gen2) once the XVE PLMs are HS-opened; aligning the remaining link knobs to Gen2 and retraining from the root port yields a live Gen2 link at the native lane width. Gen3 resisted our attempts: the Gen3-disable fuse exposes no override field that we found, and the higher-generation PHY per-rate calibration appears to be fuse-gated and is never run in the sequences we observed, so the partial table edits we tried did not bring up Gen3. We stress *appears* and *we found*: our understanding of the link initialization is incomplete, and we cannot exclude a Gen3 path that a fuller account of the boot sequence would expose. Gen2 is the link-speed ceiling we reached in firmware, not one we proved. The lane width, by contrast, is a confirmed *physical* gate rather than a firmware one: the card trains at $\times 4$ because the AC-coupling capacitors for the upper twelve lanes are depopulated on the board, and we restored a full $\times 16$ link by hand-soldering the 24 missing capacitors. That is a board rework outside the firmware threat model, and it is the cleanest illustration of the boundary this paper draws: where differentiation is enforced in the silicon or on the PCB, the firmware break does not move it, and only a soldering iron does.

6.5 The boundary, stated cleanly

The taxonomy (Table 1):

- **Defeated in firmware** = anything we found to be enforced by a *fuse shadow register read by firmware*: SM rate, capacity, PCIe Gen2. These we demonstrate.
- **Defeated only in hardware** = lane width, a *confirmed* physical gate: the upper lanes are routed but their AC-coupling capacitors are depopulated, and we brought up a full $\times 16$ link by soldering the 24 missing capacitors. The firmware break does not touch this axis; a board rework does. It marks the edge of what software alone can reach.
- **Not defeated (by us)** = the axes our attempts did not crack: PCIe Gen3, the apparent fuse-gated PHY calibration, and the HBM controller’s mode-register path (and hence on-die ECC), which appears source-ID-locked. The labels in this row are our *working hypotheses* for why each resisted, not established mechanism: we did not fully reverse the fuse semantics or the boot initialization, so each is an open question rather than a proven wall.

For the axes we did break, the lesson is that a differentiation scheme is only as strong as its weakest “firmware reads a fuse and decides” check. Whether the remaining axes are genuinely stronger or merely better hidden from the effort we spent, we leave open.

Table 2: De-differentiation results per axis. Native vs. unlocked, factor, stability, persistence, and the price tier the unlocked configuration corresponds to. Source: benchmark log set and 80 GB memory A/B runs (see verification appendix).

Axis	Native (crippled)	Factor	Unlocked	Stability	Persistence
FP32 SGEMM	393 Gflop/s	$\sim 31\times$	12,233–12,256 Gflop/s	safe, 67 °C	FLR + reload
FP64 DGEMM	197 Gflop/s	$\sim 59\times$	11,668 Gflop/s	safe	FLR + reload
FP64 tensor	191 Gflop/s	$\sim 62\times$	11,786 Gflop/s	safe, 64 °C	FLR + reload
TF32 tensor	6,242 Gflop/s	$\sim 15\times$	94,103 Gflop/s (94 TFLOPS)	safe, 64 °C	FLR + reload
Memory capacity	10 GB	$8\times$	80 GB	0 err @ 64.6 TFLOPS ^a	reload
PCIe link	Gen1 $\times 4$	$2\times$ BW	Gen2 $\times 4$	live, retrained	FLR

^aThe full 80 GB is physically present (hash write/read-back) but shows errors under stress (2,796) at the stock refresh interval while running cleanly at 40 GB, consistent with a QC-binned part; refreshing more often yields 0 errors at $\sim 32\%$ throughput cost. Full-speed Gflop/s are the benchmark’s own estimate; the robust result is the native→unlocked jump.

7. Evaluation

7.1 Results

Table 2 summarizes the de-differentiation results per axis: native versus unlocked, the speedup/capacity/bandwidth factor, stability (error counts where relevant), persistence (survival of FLR and driver reload), and the price tier the unlocked configuration corresponds to. The numbers are drawn from the SM benchmark log set and the memory-stability A/B runs.

Across the three defeated axes we measured roughly $31\times$ FP32 and $59\text{--}62\times$ FP64 compute, $8\times$ capacity (10 → 80 GB) at a stable 64.6 TFLOPS, and $2\times$ link bandwidth (Gen1 → Gen2). The compute and capacity gains survive a driver reload, because the override register values themselves persist across FLR.

7.2 Reproducibility, robustness, and a regression finding

We examined a cross-version driver corpus spanning roughly 2020 to 2025: branches 450, 460, 470, 510 (two point releases), 515, 525, 535, 560, 570, and 580. The 580 branch also supplies the open-kernel-modules source.

The unbounded sig-read DMA → HS code-exec primitive of §5 is a **regression**. In the older booter images in this corpus the overflow is **absent**; in the 580 open-kernel-modules-era booter [16] it is **present** (the signature-verification routine issuing the DMA-copy helper with no bound on the adversary-controlled length). We verified this offline by recovering and disassembling each version’s booter where we could: the 510 SEC2 booter was analyzed in full, and its signature path uses only constant or clamped DMA lengths, with no metadata-sized copy, while the 580 booter exhibits the unbounded copy, matching the hardware-verified primitive.

We do **not** claim a precise first-affected branch. The packaging of the booter image changed in the newer firmware, and we could not recover the 525 image for inspection; 515 through 570 remain indeterminate pending further work. In short, the overflow is absent in 510, present in 580, and indeterminate for 515 through 570.

The regression has two implications. First, a newer firmware version is not necessarily more hardened, so advising users to stay current is not by itself a security argument. Second, this defect entered during the open-source-driver era, when one might expect broader review. Open-sourcing the kernel modules did not surface a textbook unbounded-DMA stack smash in the signed binary booter those modules load. We return to both points in §8.7.

7.3 Cost

The owner’s cost is a commodity card, host root, and the primitives in this paper, with no special equipment, no signing key, no physical access. Set against the SKU price delta between the crippled part and the

configuration it unlocks to, the economic asymmetry is large: a one-time software exploit moves the part across a multi-thousand-dollar price tier, and the change persists across reboots of the driver.

8. Discussion and Lessons

8.1 The canary debate is on the wrong axis (Thesis 1)

Hardening guidance fixates on canary entropy and secrecy. This case is a strong counterexample to that emphasis: the guard was hardware-secure-RNG-seeded and per-boot fresh, everything the entropy axis asks for, and it provided no protection at all. If the reference word is writable and reachable, entropy buys nothing: the adversary normalizes the guard and the saved copy to the same value in one write and never observes the random secret. Recommendations: (a) place the stack-guard global in a region made **read-only after init** (a RELRO-equivalent) or behind an **MPU read-only mapping**, which is mandatory in flat-memory firmware that has no MMU isolation between globals; (b) separate truly read-only data from writable globals in the link script, and never let the guard land adjacent to overflowable buffers; (c) in signed/flat firmware, SSP provides near-zero value by default, so pair it with control-flow integrity [9], shadow stacks, or hardware stack limits. This is a toolchain- and linker-script-default footgun for the entire signed-firmware ecosystem, independent of vendor or ISA.

8.2 DMA opacity (Thesis 3), evidenced by the vendor’s own miss

Signed-image review and supply-chain attestation give a false sense of completeness, because post-DMA reachability is invisible to static tooling. The vendor’s own miss shows this: the only way an unbounded DMA copy with an adversary-controlled length reaches release in newer code is if the analysis pipeline does not treat the DMA engine as a write primitive. DMA writes are modeled as opaque transfers, not as bytes-into-a-bounded-buffer that can clobber the canary and the return address. Recommendations: transport-aware static modeling (treat every DMA sink as a length-checked copy with adversary-controlled length until proven otherwise), runtime attestation of DMA’d control data, and a release gate that diffs every DMA sink against the prior image. The same opacity cut both ways: it kept some of our own attacks (runtime HBM retiming) out of reach, and it hid this bug from the vendor.

8.3 Minimal firmware is easier to exploit, not harder (Thesis 2)

The standard guidance (minimize the trusted computing base, no OS in the secure coprocessor) reduces *bug count* but *maximizes predictability*, and predictability is what makes memory-corruption exploits surgical and reliable. The minimal, deterministic booter is what made offline emulation a reliable oracle. Recommendations: inject nondeterminism into security-critical paths where feasible (randomized stack base, guard-page faults, CFI), and assume the adversary has a perfect emulator of any shipped deterministic image, then design mitigations to survive that assumption. The emulator is dual-use; it also enabled responsible analysis, mirroring §8.2.

8.4 Fuses are not a security boundary against firmware

When a fuse is only *read* by subvertible firmware, it differentiates *price* but not *security*, and we showed three such fuses fall. Real permanence would require the fuse to *physically gate* the datapath; PCIe Gen3, the PHY calibration, and the HBM controller’s source-ID behavior are our *candidates* for that stronger category, but we did not establish that any of them truly is one (they resisted us; we did not prove they are unbreakable). The actionable point stands regardless: vendors should classify each fuse and document which tier each SKU difference sits in; a difference enforced only by a firmware fuse-read should not be marketed as hardware-permanent.

8.5 Persistence across FLR

Override values held in an always-on island are an under-appreciated risk: once written, they turn a transient, one-shot exploit into a durable unlocked state that survives a function-level reset and a clean stock driver

load. The exploit only needs to run once, and the override persists afterward. Defenders should audit which security-relevant state lives in always-on islands and whether it is re-validated on reset.

8.6 Toward DMA-aware static analysis

This bug class deserves its own defensive treatment, because conventional analyses are *structurally* unable to see it.

Root cause. The overflow is a write of adversary-controlled length into a fixed-extent buffer, but the store is **not a CPU instruction**: it is effected by a DMA engine programmed through memory-mapped/coprocessor registers (latch source, destination, and length; write a trigger). Conventional overflow and taint analyses track CPU stores and known library sinks. A DMA transfer is a few register writes plus a poll, so the write into the signature buffer is invisible to the tool, the length operand is never related to the buffer extent, and the canary smash falls outside the tool’s memory-effect model. Worse, the length is a field parsed from a struct that was *itself* DMA’d from untrusted system memory, so the taint *source* is also a DMA. Three modeling gaps: a DMA **write** is not a recognized effect; a DMA **read** from untrusted memory is not a recognized taint source; and the **buffer extent** in flat memory has no allocator metadata to bound against.

Six additions to a static pipeline would close these gaps.

- **(A) Lift device effects into the IR as DMA-as-copy summaries.** A per-engine semantic model: latching (source, destination, length) plus a trigger write is equivalent to a copy of **length** bytes. The work is recognizing the programming idiom (the coprocessor master window, the engine’s DMA opcodes) and attaching the summary, exactly as analyzers special-case the standard memory-copy routine. Every downstream technique then works unchanged.
- **(B) Taint with DMA as a source.** Mark buffers filled by a DMA from host/system-memory addresses as adversary-tainted, and propagate through struct parsing so the length load inherits taint. This needs a field-provenance/shape pass; the relevant struct layouts are already in the firmware headers.
- **(C) Bounded-write check at DMA sinks.** For a DMA write to a buffer of extent S at offset o with length L , discharge $L \leq S - o$; if L is tainted and unconstrained by a dominating guard, report. Value-set analysis [5] or query-based dataflow with the DMA sink as a custom sink suffices, and path-sensitivity makes it precise: fire only when no dominating bound exists, which is exactly what makes it a clean **regression** signal, since the older firmware bounded the length.
- **(D) Layout/adjacency escalation, link-map-aware.** Flat memory has no allocation header, so feed the analyzer the link map: infer extent as distance-to-next-object, and reason about *what the overrun reaches* (guard global $\rightarrow$ saved canary copies $\rightarrow$ return addresses) to escalate “overflow” to “canary defeat / control-flow hijack.” Independently reportable: a *writable stack guard within DMA-overrun reach of any tainted-length sink* is a design defect regardless of a specific proof.
- **(E) Contracts on the DMA HAL.** The DMA copy entry point is a natural contract boundary; a **requires** `len  $\leq$  extent(dst)` precondition verified at each call site reduces the whole class to a local obligation and documents intent, so a later revision cannot silently widen it. This is the cheapest durable fix when source is available.
- **(F) Hybrid analysis plus release gating.** DMA-aware symbolic execution (model the engine, make the length symbolic, solve the overrun threshold) reproduces mechanically what our emulator found empirically. The highest value-per-effort defense is a **release gate** that enumerates every DMA sink and diffs the *bound provenance* of its length operand: a length that went from constant/checked to tainted-unbounded is exactly this bug.

Limitations. Engine models are proprietary, but the vendor has the documentation, so the gap is process rather than feasibility. Indirect register-programming idiom recognition is imperfect under aliasing; extent inference needs link-map inputs analyzers do not usually consume; and false positives on legitimately variable DMAs require the same path-sensitive bound recognition as the regression detector.

Firmware-rehosting research already builds memory-mapped-I/O and DMA peripheral models for dynamic analysis: P2IM [1], HALucinator [2], Fuzzware [3], and especially DICE [4], which automatically emulates DMA input channels. Those models are built for fuzzing. The same device models would serve a static or abstract-interpretation pipeline and a release gate, where they would make DMA writes a first-class memory

effect at review time rather than only at fuzz time. The capability already exists in the research community; it is simply absent from this vendor’s signed-firmware review, as the shipped regression shows.

We implemented the first four layers as a small static checker over the booter’s instruction stream and ran it across the firmware corpus. It models the engine’s transfer as a copy sink by recognising the programming idiom (a fixed destination buffer latched immediately before the call into the routine that issues the transfer) and finds that routine automatically per image, without any hard-coded address, as a call target whose body reaches the engine’s transfer opcodes and that is repeatedly invoked with that fixed-destination shape. For each such routine it recovers the calling convention from the chunk loop’s “remaining = total – transferred” arithmetic, which tells it which argument is the length. It then treats a length read from a field of a structure that was itself transferred in from untrusted memory (recognised by that structure being parsed at several distinct field offsets) as adversary-tainted, and discharges the bounded-write obligation: a length that is a compile-time constant, a fixed encoded transfer size, or capped by a dominating compare/clamp is safe; a tainted length reaching the transfer with no dominating bound is reported. Run as a differential gate across versions, the same checker flags the open-kernel-era booter’s signature-read transfer, the unbounded copy into the fixed signature buffer that our primitive exploits, as its single unbounded sink, and passes the older booter, whose corresponding lengths are constant or clamped, with **no false positives** across that older family. Images we could not recover, or whose relevant routine we could not isolate, are reported as *indeterminate* rather than clean, so no version boundary is overclaimed. The limitations are unchanged: per-image idiom recognition is heuristic, extent inference still wants link-map inputs, and the reasoning is confined to a single procedure. Within those bounds it still detects the regression mechanically and without manual guidance.

8.7 A regression: newer, open-source firmware was not safer here

The HS code-exec primitive is a regression (§7.2): older booters lack the unbounded DMA; the newer open-kernel-modules booter introduced it. Two lessons follow. First, threat models that assume “patch to the latest firmware” as mitigation are unsound when newer images add new memory-corruption surface, so version currency is not a security gradient. Second, open-sourcing the surrounding kernel modules created an expectation of broader scrutiny that did **not** extend to the signed binary booter those modules load; the most safety-critical component (HS entry) stayed a closed blob and shipped a textbook unbounded-DMA stack smash. We recommend treating signed-firmware blobs as first-class review targets even (and especially) when the host driver is open, and gating releases on differential analysis against the prior image’s memory-safety-relevant call sites (every DMA sink with a caller-supplied length).

9. Related Work

Falcon / HS reverse engineering and secure boot. Community work (the envytools lineage [12]) mapped the Falcon ISA and early secure-boot mechanics; prior secure-boot analyses on game consoles and mobile SoCs established the pattern of attacking memory safety in code that runs once a signature check has passed. Our contribution sits downstream of signature verification: we load the signed code unchanged and corrupt its state while it runs.

Stack-canary weaknesses. Prior art attacks the canary’s *value*: fork-server byte-by-byte brute force (e.g. against forking network daemons) [7], info-leak-then-replay, and the recurring weakness of fixed embedded canaries. Our framing is different and arguably more basic: a reference-integrity break makes the value irrelevant, defeating even a maximal-entropy, per-boot, secret canary. We are not aware of prior work that shows a best-practice randomized canary providing no protection purely because of where the reference word is placed in a shipped secure firmware.

Firmware rehosting and peripheral modeling. P2IM [1], HALucinator [2], Fuzzware [3], and DICE [4] build MMIO/DMA peripheral models for dynamic analysis and fuzzing. We borrow their device-modeling insight but argue for moving it into *static* review and release gating (§8.6), and we use a bespoke deterministic emulator as a *state oracle* rather than a fuzzing harness.

Static memory-safety analysis. Value-set analysis (Balakrishnan and Reps) [5] and modern query-based dataflow are the natural carriers for the bounded-write check we propose; the missing piece is the DMA effect model.

GPU security. Prior work covers GPU-resident Rowhammer [11], GPU microarchitectural side channels [10], and memory-isolation gaps. Our target is different: the firmware trust boundary that enforces commercial differentiation.

Fuse and feature-unlock culture. CPU microcode patching, console and phone bootloader unlocks, and soft-strap SKU lifting are the cultural precedents. Most directly, Xing [13] studies the same CMP 170HX and recovers compute at the application layer: by routing around the throttled fused-multiply-add path through CUDA source changes, on stock firmware, reporting over $15\times$ FP32 and over $3\times$ large-language-model inference. That method avoids the throttle from software running on top of the firmware, whereas we remove it underneath by defeating the fuse that imposes it, which restores the fused rate itself rather than avoiding the slow pipe, and reaches axes an application-level workaround cannot, namely the memory capacity and the PCIe link. Relative to all of this, our contribution is the combination: a value-independent HS canary break, emulator-driven continuation authoring, an economic-differentiation target, and the DMA-opacity systematization.

10. Ethics and Disclosure

This was lab-only work on a single card, with no resale and no persistent silicon change. We extracted no signing keys and forged no signature; the unlock state is non-persistent across a power cycle that clears the always-on island, and we restored the card to its native configuration after measurement.

We disclose this work in full and in public, and we notify the vendor’s product-security team concurrently with publication. We deliberately do not offer a pre-publication embargo, and we want to be explicit about why, since it departs from the usual coordinated-disclosure norm. Coordinated disclosure rests on an assumption that does not hold for this class of defect: that the vendor’s remedy protects the user. The remedy here is not a patch that shields an owner from an attacker. It is an irreversible, fuse-enforced revocation of a capability that benefits the owner, applied to the owner. A private embargo window would let the vendor burn the relevant anti-rollback fuses on already-shipped hardware, permanently removing that capability from the very users this work concerns, before those users could learn of it or act. In the inverted threat model of this paper, where the defender is the device and the adversary is its owner, declining the embargo is the user-protective choice rather than a departure from it: public, simultaneous disclosure gives the vendor and the public the same information at the same time and pre-empts no one.

Where the same underlying memory-safety defect could reach a product whose threat model trusts the tenant over the host, such as a confidential-computing [15] or multi-tenant deployment, a fix would protect a genuine third party. Our public disclosure informs the vendor of the defect so they can evaluate and address those cases; we withhold only the private head-start, not the information itself.

We frame the work as legitimate security research and as an interoperability and right-to-repair investigation, and we are mindful of the tension with warranty and terms-of-service constraints on feature unlocking. We publish the lessons (canary reference-integrity, DMA opacity, fuse tiering, and determinism as an exploitation oracle) because they apply to any firmware vendor, not only the one whose part we examined. The defensive recommendations of Section 8 are the intended takeaway.

11. Conclusion

The product-segmentation boundary on this card rested on a stack canary and a signature check, and we got past both without breaking either directly. The canary was randomized and the signature held; what let us through was the placement of the canary’s reference word in writable memory, together with a deterministic target that an emulator can reproduce closely enough to author a coherent post-overflow continuation. With

that foothold we unlocked the SM math rate, the memory capacity, and the PCIe link up to Gen2, while two further axes resisted the attempts we made and one more, the lane width, yielded only to a board rework rather than to firmware.

The specific card will be superseded, but four lessons should carry over. The integrity of a canary’s reference word matters more than the entropy of its value. Static review has to treat DMA as a first-class memory effect, since the vendor’s own miss shows that it currently does not. A deterministic, minimal target is easier to exploit because it is easy to emulate. And a fuse that is only read by firmware is a weaker boundary than one that physically gates a datapath.

References

1. P2IM: Scalable and Hardware-Independent Firmware Testing via Automatic Peripheral Interface Modeling. B. Feng, A. Mera, L. Lu. *USENIX Security* 2020.
2. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. A. A. Clements, E. Gustafson, T. Scharnowski, et al. *USENIX Security* 2020.
3. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. T. Scharnowski, N. Bars, M. Schloegel, et al. *USENIX Security* 2022.
4. DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis. A. Mera, B. Feng, L. Lu, E. Kirda, W. Robertson. *IEEE Symposium on Security and Privacy (S&P)* 2021.
5. Analyzing Memory Accesses in x86 Executables (value-set analysis). G. Balakrishnan, T. Reps. *International Conference on Compiler Construction (CC)* 2004.
6. StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks. C. Cowan, C. Pu, D. Maier, et al. *USENIX Security* 1998.
7. Hacking Blind (Blind Return-Oriented Programming, BROP). A. Bittau, A. Belay, A. Mashtizadeh, D. Mazières, D. Boneh. *IEEE Symposium on Security and Privacy (S&P)* 2014.
8. The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (Return-Oriented Programming). H. Shacham. *ACM CCS* 2007.
9. Control-Flow Integrity. M. Abadi, M. Budiu, Ú. Erlingsson, J. Ligatti. *ACM CCS* 2005.
10. Rendered Insecure: GPU Side Channel Attacks are Practical. H. Naghibijouybari, A. Neupane, Z. Qian, N. Abu-Ghazaleh. *ACM CCS* 2018.
11. Grand Pwning Unit: Accelerating Microarchitectural Attacks with the GPU (GPU-driven Rowhammer). P. Frigo, C. Giuffrida, H. Bos, K. Razavi. *IEEE Symposium on Security and Privacy (S&P)* 2018.
12. envytools: Tools and documentation for reverse-engineering NVIDIA GPUs (Falcon ISA and secure-boot documentation). M. Koscielnicki et al. Open-source project.
13. Exploration of Cryptocurrency Mining-Specific GPUs in AI Applications: A Case Study of CMP 170HX. K. Xing. *arXiv:2505.03782*, 2025.
14. gpu-burn: a multi-GPU CUDA stress test. V. Timonen. Open-source tool, github.com/wilicc/gpu-burn.
15. NVIDIA H100 Tensor Core GPU Architecture, including Confidential Computing. NVIDIA whitepaper, 2022.
16. NVIDIA Open GPU Kernel Modules. NVIDIA. github.com/NVIDIA/open-gpu-kernel-modules, 2022.